

Testing & Verification Guide

This guide outlines the procedure for running end-to-end integration tests, capturing and decrypting network traffic using `tshark` /Wireshark, and checking transport telemetry.

1. Quick Local Test

Run both the server and client on `localhost` to verify the compilation, parser, and reassembler pipeline.

Step 1: Generate TLS Credentials

Ensure you have generated the required self-signed certificates:

```
mkdir -p certs
openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 \
  -keyout certs/server.key -out certs/server.crt \
  -subj "/CN=localhost"
```

Step 2: Launch the Server

Start the server first to listen on port `4433` and specify the output filename:

```
cd build
./moq_server 0.0.0.0 4433 output_test.h265 ../certs/server.crt ../certs/server.key
```

Step 3: Run the Client

In another terminal, stream the source H.265 video:

```
cd build
./moq_client 127.0.0.1 4433 input.h265 2 1200
```

- `2` : Total streams (Stream 0 for critical, Stream 1 for P-frames).
- `1200` : Maximum payload chunk size in bytes.

Step 4: Verify Output Integrity

Compare the transferred file against the original input using `cmp` or `md5sum` . They must be identical:

```
# Wait for the client to print "Client finished cleanly"
cmp input.h265 output_test.h265
echo $? # Output should be 0 (meaning files are identical)
```

2. Multi Machines Connection Test

To test media streaming between two different physical machines or virtual machines in a real network environment:

Prerequisites

- Identify Host A (Server) and Host B (Client) IP addresses (e.g., Host A IP is `192.168.1.10`).
- Ensure both hosts are on the same subnet or connected via VPN, and can ping each other.

Step 1: Server Setup (Host A - IP: `192.168.1.10`)

Generate self-signed TLS credentials on Host A and launch the server:

```
# Bind listener to all interfaces
./moq_server 0.0.0.0 4433 output_test.h265 certs/server.crt certs/server.key
```

Step 2: Client Setup (Host B)

Launch the client on Host B, pointing directly to Host A's IP address:

```
./moq_client 192.168.1.10 4433 input.h265 2 1200
```

Step 3: Verification

Transfer the reassembled `output_test.h265` from Host A back to Host B and compare hashes:

```
scp user@192.168.1.10:/path/to/output_test.h265 ./
cmp input.h265 output_test.h265
```

3. Concurrent Multi-Connection Test

Testing multiple concurrent clients is achieved by running multiple server processes on different ports and connecting distinct client processes simultaneously.

Step 1: Launch Multiple Servers on Different Ports

Start server processes in the background or in separate terminal tabs, assigning a unique port and output filename for each:

```
cd build

./moq_server 0.0.0.0 4433 output_33.h265 ../certs/server.crt ../certs/server.key &
./moq_server 0.0.0.0 4434 output_34.h265 ../certs/server.crt ../certs/server.key &
```

Step 2: Run Multiple Clients Concurrently

In your client terminal, launch client instances simultaneously targeting the respective server ports:

```
cd build

./moq_client 127.0.0.1 4433 input.h265 2 1200 &
./moq_client 127.0.0.1 4434 input.h265 2 1200 &
```

Step 3: Verify Output Integrity

Step 3: Verify Output Integrity

After both client processes complete, verify that both output files match the original input file:

```
cmp input.h265 output_33.h265
cmp input.h265 output_34.h265
```

(Note: You can terminate background server processes using `kill moq_server`).

4. Telemetry Verification & Console Log Analysis

During runtime, both the client and server print detailed logs capturing internal packet states and transport events. Below is an explanation of the most important log statements.

Client Logs

1. Connection Readiness Logs

```
[cli] wait_connected...
[cli] Connected
[cli] Streams ready
```

- **Explanation:** The client successfully completed the QUIC handshake and TLS negotiation, and initialized all target streams.

2. Fragment Send Completion Logs

```
[cli] SEND_COMPLETE nalu_id=142 frag_idx=3
```

- **Explanation:** MSQuic has successfully pushed fragment `3` of NALU `142` onto the physical network card buffer. The client's local memory mapped to this fragment has been freed.

3. Client Shutdown Sequence

```
Client finished cleanly
Total Application-Layer packets sent: 8251
[cli] SHUTDOWN_COMPLETE
```

- **Explanation:** All media fragments were sent. The client initiated a graceful teardown, verified all streams received peer ACKs, and terminated cleanly.

Server Logs

1. Parsing & Stream Assembly Logs

```
[parse] nalu_id=142 payload_len=1200
```

- **Explanation:** The server's stream framing parser read a 16-byte `PacketHeader` from the QUIC stream, indicating that this incoming payload belongs to NALU `142` and has a chunk size of `1200` bytes.

2. Fragment Reception Order Logs

```
[RECV_ORDER] t=1034188129754 nalu=142 frag=3/5 bytes=1200
```

- **Explanation:** The Reassembler received fragment index **3** (out of **5** total fragments) of Nalu **142**, with a size of **1200** bytes, at local monotonic timestamp **1034188129754** microseconds.

3. Reassembly Completion & Write-out Logs

```
[srv] about to complete consumed=0 total=5640  
[srv] COMPLETE CALLED
```

- **Explanation:** All **5** fragments for Nalu **142** have arrived. The Reassembler joined them into a contiguous **5640** byte buffer and written/flushed it to the file output stream.

4. Connection Termination Logs

```
[srv] PEER_SEND_SHUTDOWN stream=0x7f03a4002700  
[srv] SHUTDOWN_COMPLETE
```

- **Explanation:** The server received the client's graceful stream shutdown signal (**FIN** bit), closed its listener connection hooks, and cleaned up session state.

Saving Console Logs to a File

To save or redirect the terminal output (both stdout and stderr) to a text file for post-analysis:

- **Method A: Redirect to file (Silent in console)**

Use the **>** redirector to capture both stdout and stderr (indicated by **2>&1**):

```
./moq_client 127.0.0.1 4433 input.h265 2 1200 > client.log 2>&1
```

- **Method B: Redirect to file and print to console concurrently (Recommended)**

Pipe the output through the **tee** utility to monitor the transfer in real-time while writing to a log file:

```
./moq_client 127.0.0.1 4433 input.h265 2 1200 2>&1 | tee client.log
```

5. Traffic Capture, Decryption & Wireshark Verification Guide

Although standard Wireshark captures can recognize the packets as QUIC protocol, because QUIC mandates TLS 1.3 encryption, you cannot inspect the inner Stream Data payloads without decrypting them first. This guide outlines how to configure Wireshark and **tshark** to decrypt and verify the H.265-over-MsQuic streaming system, ensuring the custom **PacketHeader** and video payload are correctly transmitted over the wire.

The instructions below assume you are using Wireshark on a Windows machine.

0. Install the Lua Dissector Plugin

Because the application header uses a custom format, you must register a Lua dissector script (**moq_dissector.lua**) for Wireshark to interpret the headers.

1. Open **Wireshark** on Windows.

2. Select **Help -> About Wireshark** from the top menu.
3. Select the **Folders** tab.
4. Locate the row named **Personal Lua Plugins** and **double-click the path**. Windows Explorer will open this directory.
5. In this folder, create a new file named `moq_dissector.lua`.
6. Copy the entire content of the virtual machine's dissector script located at `~/config/wireshark/plugins/moq_dissector.lua` and paste it into the new file.
7. Reload the plugins in Wireshark by pressing `Ctrl+Shift+L` (or restart Wireshark).

Step 1: Prepare and Start Background Capture (Terminal A)

On the server host, launch the server process and begin `tshark` capture in the background:

```
cd build

# 1. Start the server (optionally set LD_LIBRARY_PATH to point to debug MsQuic bin directory)
LD_LIBRARY_PATH=~/.msquic/build/bin/Debug ./moq_server 0.0.0.0 4433 /tmp/out.h265 ../certs/server.crt ..

# 2. Start tshark background capture
# Note: Replace interface 'lo' with 'eth0' or 'any' if capturing across external networks
sudo tshark -i lo -f "udp port 4433" -w /tmp/ultimate_capture.pcap
```

Step 2: Stream Data and Export TLS Decryption Keys (Terminal B)

On the client host, run `moq_client` while exporting the TLS decryption secrets using the `SSLKEYLOGFILE` environment variable:

```
cd build
rm -f /tmp/ultimate_keys.log

# Run the client and write key log details (Please note the correct argument order)
LD_LIBRARY_PATH=~/.msquic/build/bin/Debug \
SSLKEYLOGFILE=/tmp/ultimate_keys.log \
./moq_client 127.0.0.1 4433 input.h265 2 1200
```

This forces the TLS layer to write the keys to `/tmp/ultimate_keys.log`.

Step 3: Stop Capture and Retrieve Files

After the client finishes, terminate both the capture and server on Terminal A:

```
# 1. Stop tshark with Ctrl+C
# 2. Kill the background server instance
pkill -f moq_server
```

Transfer the generated capture files to your local workstation for analysis:

1. `/tmp/ultimate_capture.pcap` (The raw packet capture)
2. `/tmp/ultimate_keys.log` (The TLS session keys)

Wireshark Verification Methods

1. Load Keys and Decrypt

1. Open Wireshark and open `ultimate_capture.pcap`.
2. Go to **Edit -> Preferences -> Protocols -> TLS**.
3. Set the **(Pre)-Master-Secret log filename** to point to your copied `ultimate_keys.log` file.
4. Click **OK**.
5. Select a QUIC packet; you should now see **Protected Payload (decrypted)** or **Stream Data** in the details section, confirming decryption is active.

2. QUIC Stream Segmentation & Alignment Notes

QUIC acts as a continuous byte-stream. Because of physical MTU constraints, application-layer chunks **will get fragmented and misaligned with physical QUIC STREAM frames**:

- The custom Lua dissector plugin will only parse and show headers that align exactly with the start of a decrypted QUIC **STREAM** frame (i.e. `offset=0` or at alignment boundaries).
- Most of the time, the 16-byte header is split across multiple UDP packets.

3. Methods to Inspect Custom Headers

1. Type `moq` in the Display Filter bar at the top of Wireshark and press Enter.
2. The remaining packets are those containing complete, aligned custom headers.
3. Select one of the filtered packets and expand the **MoQ PacketHeader** detail pane to view the fields (`nalu_id` , `payload_len` , `fragment_idx` , `fragment_count`) parsed in plain text.